



Federated learning based on two-stage knowledge distillation for intrusion detection in industrial IoT

Renqiang Zhou ^a, Zhendong Wang ^{a,*}, Shuxin Yang ^a, Daojing He ^{b,c}, Sammy Chan ^d

^a School of Information Engineering, Jiangxi University of Science and Technology, Ganzhou, 341000, Jiangxi, China

^b Software Engineering Institute, Nanchang University, Nanchang, 330047, Jiangxi, China

^c School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, 518055, Guangdong, China

^d Department of Electrical Engineering, City University of Hong Kong, Hong Kong, China

ARTICLE INFO

Keywords:

Intrusion detection
Industrial IoT
Federated learning
Knowledge distillation
Privacy preservation

ABSTRACT

Considering that the deployment of intrusion detection systems (IDS) in industrial Internet-of-Things (IIoT) environments still faces resource constraints as well as customer data sensitivity, privacy and other challenges, this paper proposes a federated transfer learning framework based on two-stage knowledge distillation (FTKD). The FTKD approach combines personalized knowledge distillation (PKD) and adaptive progressive self-Knowledge distillation (APSKD) by designing a lightweight MA-TBCLNN model to facilitate the learning process of local model personalization. Moreover, the FTKD method adopts an aggregation strategy based on Shannon entropy and maximum mean difference (EMMD), which further improves the robustness and adaptability of the model under diverse data characteristics. To enhance privacy protection, the FTKD method uses Logits, the predicted values of local models on public datasets, as the aggregation object, and introduces the uniform perturbation mechanism to achieve differential privacy protection. The experimental results show that the FTKD method achieves greater than 97% accuracy on both ToN-IIoT and Edge-IIoTset datasets, which significantly outperforms existing federated learning methods and intrusion detection methods.

1. Introduction

Under the wave of digital transformation, the Industrial Internet of Things (IIoT) as a bridge connecting the physical and digital worlds, is reshaping the global industrial landscape at an unprecedented speed. However, this interconnectivity also brings unprecedented security challenges, making network infrastructure a coveted target for malicious actors. According to statistics, cyber-attacks against IIoT systems have been frequent in recent years, posing a serious threat to personal privacy, corporate assets, and even national security (Xenofontos et al., 2022).

To address with the increasingly complex network security threats, intrusion detection systems (IDS) have emerged as one of the key lines of defense to safeguard network security. IDS are mainly divided into host intrusion detection systems (HIDS) and network intrusion detection systems (NIDS). HIDS are installed directly on the hosts to be protected, and they ensure host-level security by monitoring operating system logs, file changes, and other system behaviors (Liu et al., 2018). This approach is particularly effective for identifying insider threats targeting specific systems, but the applicability of HID in large-scale network environ-

ments is limited because of high resource consumption, high maintenance costs, and the ability to protect only their own independent systems. In contrast, NID is particularly adept at identifying external and distributed attacks by listening to traffic across the entire network and being able to capture activity patterns across multiple hosts (He et al., 2023). Taking advantage of its global view, NID not only detects known threats, but also discovers new types of attacks based on the analysis of anomalous behaviors, and excels especially in the network environments of large-scale enterprises or internet service providers (ISP). With the integration of advanced technologies such as deep learning, NID is evolving towards higher intelligence and automation, further strengthening its security protection role in complex network ecology.

However, deploying advanced IDS to edge devices in IIoT is not an easy task, especially in environments with limited and decentralized processing resources. Currently, there are two main architectures for edge-deployed IDS: distributed and centralized. Centralized architectures, on the other hand, rely on a central node to aggregate information and make decisions, which simplifies management but may introduce the risk of a single point of failure (Rahman et al., 2020), whereas distributed architectures allow multiple edge devices to work together,

* Corresponding author.

E-mail addresses: 6120230744@mail.jxust.edu.cn (R. Zhou), 9120140106@jxust.edu.cn (Z. Wang), yangshuxin@jxust.edu.cn (S. Yang), eeschan@cityu.edu.hk (S. Chan).

<https://doi.org/10.1016/j.eswa.2025.130144>

Received 26 May 2025; Received in revised form 24 August 2025; Accepted 20 October 2025

Available online 28 October 2025

0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

each responsible for security monitoring in a localized area, which enhances the robustness and scalability of the system.

Federated learning, as an emerging technology framework, has great potential for addressing the above challenges. It allows edge devices distributed in different locations to collaboratively train models while minimizing the amount of data transmission and protecting sensitive information (Kairouz et al., 2021). However, federated learning for anomaly intrusion detection in IIoT scenarios still faces many challenges, among which the key challenges are as follows:

- (1) **Efficient construction of personalized client models:** in IIoT environments, different devices may encounter specific types of attacks, resulting in significant differences in their attack data. If a single global model is used to cover all the scenarios, it is likely to ignore certain device-specific attack signs, reducing overall detection efficiency and accuracy. Moreover, edge devices are often limited by computational, storage and energy constraints and cannot support complex machine learning models running directly on them.
- (2) **Non-Independently identically distributed data:** the problem of non-independently identically distributed data (non-IID) is particularly prominent within a federated learning framework, where there are significant differences between the local datasets held by different clients or devices participating in federated learning.
- (3) **Privacy protection:** in a federated learning environment, an attacker can attempt to reconstruct the dataset samples used to train the model by exploiting the model's prediction capabilities (Fredrikson et al., 2015), and may also use the model's prediction results to infer key attributes of the data samples (Yeom et al., 2018), thus posing a serious threat to individual privacy.

To address the above challenges, we propose a new framework for federated learning based on two-stage knowledge distillation (FTKD). In the FTKD approach, we propose the multi-scaled attention tied block convolution lightweight neural network (MA-TBCLNN) model. This model not only retains the lightweight advantage of the original TBCLNN model (Wang et al., 2025), but also overcomes its shortcomings in dealing with feature local details and multi-scale information, effectively enhancing the model's ability to understand complex feature dependencies. To further enhance the personalized performance of the model on each client and effectively respond to different attack scenarios, we design a two-stage knowledge distillation, i.e., a combination of personalized knowledge distillation (PKD) and adaptive progressive self-knowledge distillation (APSKD), to enhance the personalization and reliability of the local model. By combining Shannon entropy and maximum mean discrepancy (MMD), we devise the entropy-maximum mean discrepancy (EMMD) method to evaluate the distribution characteristics of the categories within the local dataset as well as the differences between the local dataset and the public dataset to adjust the weights at the time of aggregation of each client more accurately to adapt to the diverse data characteristics. In addition, we take the model-predicted probability distribution *Logits* as the aggregation object and introduce the uniform perturbation mechanism to achieve differential privacy protection. *Logits* are the last layer of activation values before the model output layer, which reflect the model's understanding and classification tendency of the given inputs to a certain extent, and they are more difficult to be used to inverse the original training data than the direct model weights. This method not only effectively avoids the exposure of sensitive information, but also ensures privacy and security throughout the communication process by adding appropriate noise, thus providing a more reliable data protection scheme for distributed learning scenarios such as federated learning.

In summary, the main contributions and innovations of this paper are as follows.

- (1) We propose a federated learning framework called FTKD, which is designed for IDS in IIoT environments. In the FTKD framework,

we propose a lightweight model named MA-TBCLNN, which overcomes its shortcomings in addressing the local details of features and multi-scale information, and effectively enhances the model's ability to understand complex feature dependencies.

- (2) We design an EMD federated aggregation strategy in FTKD, which aims to mitigating the impact of non-IID data, significantly enhancing the effectiveness of FL training.
- (3) In the FTKD framework, we introduce an innovative two-stage knowledge distillation approach including PKD and APSKD. This approach aims to dynamically allocate knowledge based on the performance of the client models and facilitates the learning process for local model personalization.
- (4) We introduce a uniform perturbation mechanism in FTKD to achieve differential privacy protection, and use the model's predictive probability distribution *Logits* as an aggregation object, which effectively avoids the exposure of sensitive information, and also ensures privacy security throughout the communication process by adding appropriate noise.

The rest of the paper is organized as follows. Section 2 reviews recent work on three topics: 1) network intrusion detection in IIoT; 2) federated learning; and 3) knowledge distillation. Section 3 describes the federated learning framework based on two-stage knowledge distillation. Section 4 analyzes the convergence of the proposed FTKD method. Section 5 describes the experimental setup and dataset as well as the preprocessing procedure. Section 6 presents the evaluation methodology and analysis of experimental results. In Section 7, the paper is summarized.

2. Related work

2.1. Network intrusion detection in IIoT

In recent years, several previous works in IIoT have demonstrated that deploying IDS is an effective way to identify threats and secure the network (Moustafa et al., 2023), and deep learning has gained wide application in the field of intrusion detection due to its powerful pattern recognition capabilities. Nandanwar and Katarya (2024) proposed a model called AttackNet, which is based on an adaptive CNN-GRU architecture, which can efficiently detect and classify different botnet attacks in industrial IoT, achieving 99.75 % accuracy on the N_BaIoT dataset. Li et al. (2024), on the other hand, proposed a hybrid DoS attack intrusion detection system, HDA-IDS, which combines feature signature-based detection with anomaly-based detection to efficiently identify both known and unknown DoS/Botnet attacks. Zhang et al. (2023) proposed the NASP-MTC method that can automatically search for the optimal neural network architecture, which can efficiently and accurately search for the optimal classification model architecture. Ding et al. (2023) proposed the DeepAK-IoT model consisting of three components: residual-based spatial representation block (RSR), temporal representation block (TRB), and detection block (DB), which in turn focuses on improving the detection accuracy of cyber-attacks through a specific structural design. Ahmad et al. (2024) proposed a deep migration learning based approach for anomaly detection in IoT and IIoT environments. The method converts data from IoT and IIoT devices into image format and uses Bagging method combined with multiple pre-trained CNN architectures to build a more robust and accurate anomaly detection model fine-tuned by optimizing hyperparameters. Obviously, the centralized deep learning approach has achieved remarkable success in the field of intrusion detection, but this approach usually requires many network traffic samples for training to achieve high detection performance. However, such datasets often contain sensitive and personal information of users, and many users are reluctant to share their data for model training due to privacy protection concerns. To resolve this contradiction, federated learning is proposed as an emerging learning framework that allows multiple parties to jointly train high-quality models without transmitting the raw

data, greatly facilitating the balance between data utilization efficiency and privacy protection.

2.2. Federated learning

Federated learning, as an innovative technique, effectively solves the problem of data silos and is able to facilitate information sharing and model training while safeguarding data privacy, and is therefore widely used in intrusion detection systems within the IIoT domain. Mothukuri et al. (2021) proposed a federated learning-based anomaly detection method that uses a gated loop unitary model to optimize the global model by sharing learned weights while maintaining user data integrity. Zainudin et al. (2023) proposed an SDN-Based model which based on FL, a framework that utilizes chi-square and Pearson's correlation coefficient feature selection methods to select potential features and reduce model complexity. Zhang et al. (2022) proposed a federated learning-based anomaly detection system for the security challenges of IIoT, which uses instance migration learning to solve the problems posed by non-IID data and introduces a ranked aggregation algorithm with a weighted voting mechanism to improve the detection Performance. Qu and Cai (2024) proposed the FEDSA framework, which improves the residual network SAREsnetV2 and constructs a novel FL framework for selecting the optimal model through global. Han et al. (2023) proposed a clustered federated meta-training framework (CFMT) incorporating meta-sampling. The framework designs a data-independent meta-sampler to balance the local dataset and employs a dynamic clustering algorithm to eliminate local models affected by non-IID data. All the aforementioned approaches propose solutions to the problems posed by non-IID data, and all rely on the direct transmission of model parameters for global model aggregation. However, there is a clear privacy risk associated with this approach, as the model parameters may leak sensitive information contained in the training data. Furthermore, while DP techniques can be applied to protect these parameters, this often sacrifices a certain level of model performance. Therefore, in this paper, we propose to achieve both user privacy protection and information sharing by passing the predictive probability distribution *Logits* of the model, while the *Logits* are noise-added through the DP technique of uniform perturbation mechanism.

2.3. Knowledge distillation

Deep learning models usually have high computational complexity and large number of parameters, which make it difficult to be directly deployed on resource-constrained IIoT devices. To address this problem, knowledge distillation (KD), as an effective model compression technique, has received much attention in the field of intrusion detection in recent years. Zhao et al. (2022) proposed an intrusion detection method based on semi-supervised federated learning (SSFL), which utilizes unlabeled data through knowledge distillation technique to enhance the classifier performance, and hard labeling strategy and voting mechanism to further reduces the communication overhead. Zhu et al. (2023) proposed a lightweight malicious traffic detection model (LKD-STNN) based on knowledge distillation, which builds a lightweight student model by means of a deeply separable convolution and a bidirectional long short-term memory network (BiLSTM). They introduced an adaptive temperature function that can dynamically adjust the temperature during knowledge transfer to obtain different softening characteristics. Wang et al. (2024b) proposed a lightweight IoT intrusion detection model (BT-TPF) based on an improved BERT-of-Theseus. The method selects a large-scale Vision Transformer as a teacher model, instructs a small-scale PoolFormer model for training, and finally uses the trained PoolFormer model as a classifier to detect network intrusion traffic. Wang et al. (2024a) proposed a lightweight intrusion detection model called TFKD. In the training phase, a teacher network containing 8-layer CNNs is pre-trained, while in the distillation phase, a single-layer CNN acts as a student network to improve the generalization

ability of the students through an adaptive temperature knowledge distillation method. However, standard knowledge distillation all require the deployment of a large teacher model, which remains challenging for resource-constrained IIoT devices. Therefore, in this paper, we design a two-stage knowledge distillation and a lightweight local deep learning model in a federated learning framework. This not only gets rid of the reliance on traditional teacher models and effectively reduces the space and memory occupation, but also enhances the personalization capability of the local model as well as its generalization and robustness to unknown attacks.

3. Proposed FTKD framework

In this section, the main focus is on introducing our proposed FTKD framework as a solution to the previously discussed challenges. The framework consists of five key approaches: (a) MA-TBCLNN model; (b) adaptive progressive self-knowledge distillation; (c) differential privacy based on uniform perturbation mechanism; (d) global aggregation; (e) personalized knowledge distillation.

Fig. 1 illustrates the FTKD framework for IIoT intrusion detection. The framework enables efficient communication between multiple IIoT clients and FL servers. By combining the proposed FTKD framework, we aim to overcome the challenges typically encountered in FL-based IIoT intrusion detection, especially the problem of client personalization models and non-IID data. To better understand the functionality of our framework, we will detail the training process of each client and the communication steps with the server side in the IIoT environment:

Step 1: Each client uses the local dataset D_i^c to train the local model by adaptive progressive self-knowledge distillation;

Step 2: After obtaining the optimal local model, use the public dataset D^p to train the model to obtain the predicted probability distribution *Logits*;

Step 3: The obtained *Logits* are noise-added via the uniform perturbation mechanism differential privacy;

Step 4: Evaluate the local dataset D_i^c using EMD and use it as the aggregation weight for each client;

Step 5: Upload the denoised *Logits* and aggregation weights to the server and get the global *Logits* by aggregating them through the *Logits* aggregator;

Step 6: The server distributes the global *Logits* to each client, and each client uses the global *Logits* as teacher *Logits* to train the local model through personalized knowledge distillation.

3.1. MA-TBCLNN model

IIoT devices in real-world applications often need to operate in resource-constrained environments that may include limited computing power, memory space, and power supply. Deploying IDSs under such conditions faces a unique set of challenges, whereas the TBCLNN model proposed in Wang et al. (2025) achieves high intrusion detection accuracy while ensuring lightweight, providing a strong technical support for solving security problems in IIoT scenarios. Based on this, we further improve the TBCLNN model by proposing the TiedMA attention mechanism and optimizing the overall architecture, thus constructing a new MA-TBCLNN model. The framework diagram of the model is shown in Fig. 2.

The overall architecture of the MA-TBCLNN model in Fig. 2 consists of several modules, including an initial convolutional layer, a lightweight residual block, a lightweight inverse residual block, and a fully connected layer. First, the initial features are extracted using depth-separable convolution, the input of this layer is a single-channel image and the output is a 16-feature map. A TiedBlockConv2d convolutional structure is used in the lightweight residual block Block 1, which has the property of residual concatenation to efficiently capture the features and minimize the loss of information. The number of output channels of

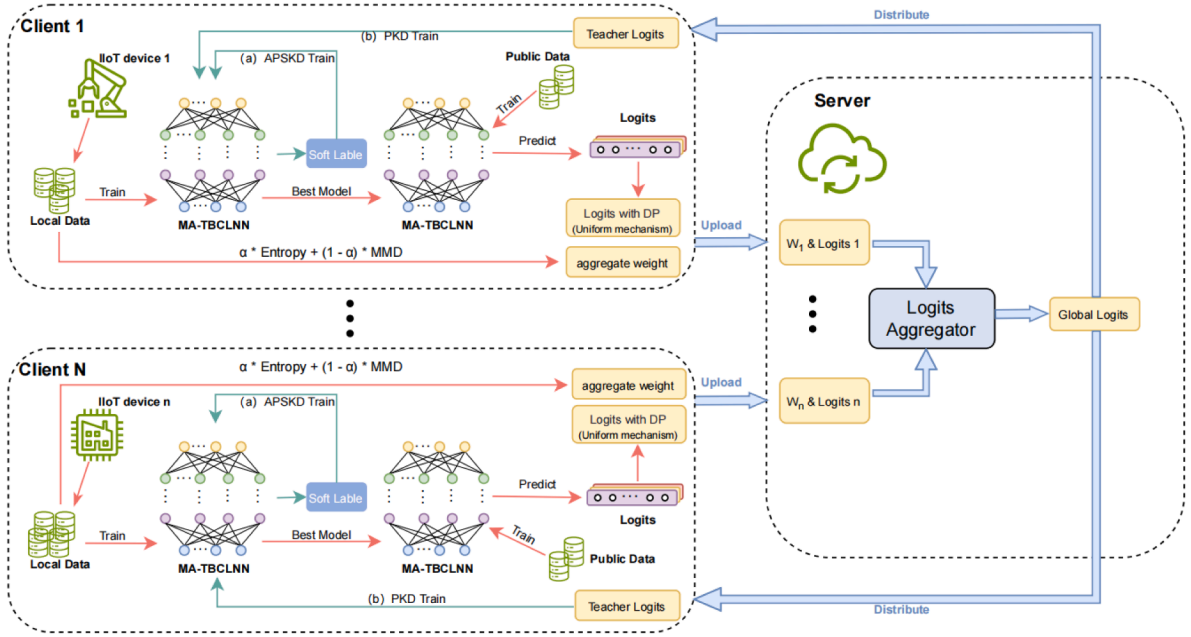


Fig. 1. Overview of the proposed FTKD framework.

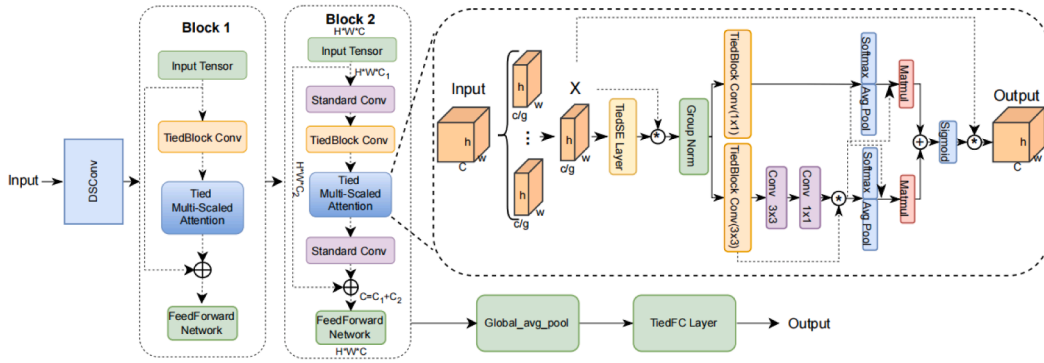


Fig. 2. Proposed framework of MA-TBCLNN Model.

this block is 32 and the step size is set to 2 to reduce the size of the feature map. The lightweight inverse residual block Block 2 further reduces the size of the feature map while increasing the number of channels, using a combination of expansion, deep convolution and compression layers to enhance the expressive power of the model. After the features are extracted, the model generates the final traffic classification results through global average pooling and TiedFC fully connected layers.

The TiedSE Layer proposed in Wang et al. (2025), while excellent at capturing inter-channel relationships, ignores spatial dimensionality information in features. To address this limitation, we propose the TiedMA attention mechanism. By introducing the spatial attention mechanism, TiedMA is able to capture spatial information effectively by paying more attention to the specific location and region of the feature map while maintaining inter-channel interactions. This approach not only helps to better preserve the importance of local features, but also ensures that key details are not overlooked, which in turn improves the overall performance and expressiveness of the model.

The overall structure of the TiedMA module is shown in Fig. 2. The module enhances the information interaction between internal channels by grouping the input features by channel dimension, with each group undergoing TiedSE layer and normalization. Two parallel branches are then used: one branch uses 1×1 tied block convolution to process global information, and the other branch uses 3×3 tied block convolution to capture local spatial information. The outputs of the two

branches are fused by matrix multiplication to generate an attention map and combined with the input features, which ultimately enhances the model's ability to capture both global and local information while reducing computational complexity.

Specifically, for a given input feature graph $X \in \mathbb{R}^{c \times h \times w}$, TiedMA divides X into G sub-features along the direction of the channel dimension, each of which is of size $\frac{c}{G} \times h \times w$, where C denotes the number of input channels, and H and W denote the spatial dimensions of the input features, respectively. Subsequently, the importance weights of each channel can be adaptively learned through the TiedSE layer. This layer performs a squeezing operation on the channels through an adaptive average pooling layer to compress the feature information of the channel dimensions into a global descriptor, and then performs an excitation operation to convert this global descriptor into the weights of the channels through a TiedFC fully-connected layer to achieve the recalibration of the importance of different channels. Two parallel paths are then utilized to extract the attention weights of the grouped feature maps, and the first branch uses 1×1 tied block convolution to process the global information. A 1×1 tied block convolution can efficiently fuse the information of different channels by performing a linear transformation over the channel dimensions to capture the global feature representation without changing the spatial dimensions of the feature maps. Next, the global attention weights are generated via the global average pooling operation and Softmax function. The global average pooling operation

is formulated as shown in Eq. 1.

$$X_{GAP} = \frac{1}{h \times w} \sum_{j=1}^w \sum_{i=1}^h X_{c/G}(i, j) \quad (1)$$

Another branch, which uses 3×3 tied block convolution captures the local space information and incorporates the local attention mechanism. The convolution kernel of a tied block convolution with a convolution kernel size of 3×3 slides over the local space and performs a convolution operation with the input sub-features to extract the feature information in the local space to obtain a preliminary local feature representation. Subsequently, this local feature representation is input to a two-dimensional convolutional layer with a convolutional kernel size of 3×3 to adjust the number of channels, which is enhanced by a ReLU activation function to enhance the nonlinear representation, and then a two-dimensional convolutional layer with a convolutional kernel size of 1×1 to recover the number of channels. After that, local attention weights are generated by a local average pooling layer and a Softmax function. Where the local average pooling operation is formulated as shown in Eq. 2.

$$X_{LAP} = \frac{1}{k^2} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{c/G}(i+m, j+n) \quad (2)$$

where k is the size of the pooling window.

After the feature representations of the two parallel paths have been obtained, the two feature representations are subjected to a matrix multiplication operation. The purpose of this matrix multiplication operation is to fuse the global and local information to generate a comprehensive attention map. Finally, the generated attention map is multiplied element-by-element with the original input feature X so that each element of the input feature is rescaled according to the weights in the attention map to obtain the final output feature.

3.2. Adaptive progressive self-Knowledge distillation

To more effectively allow the local model to draw deep knowledge from the local dataset D_i^c , we have innovatively optimized the progressive knowledge distillation technique proposed in Kim et al. (2021), and as a result, we have designed adaptive progressive self-knowledge distillation (APSKD), as shown in Fig. 3. The core idea of this approach lies in the implementation of continuous and well-targeted information feedback and guidance throughout the training cycle by allowing the student model to play the role of a teacher, fully utilizing its own learning outcomes and predicted outputs at different stages. In our approach, two loss functions are used to locally learn the student model and the teacher model of the previous epoch element, including an adaptive mutual distillation loss to transfer knowledge from the output soft labels as well as a task loss to provide task-specific supervision directly to the model of this epoch element.

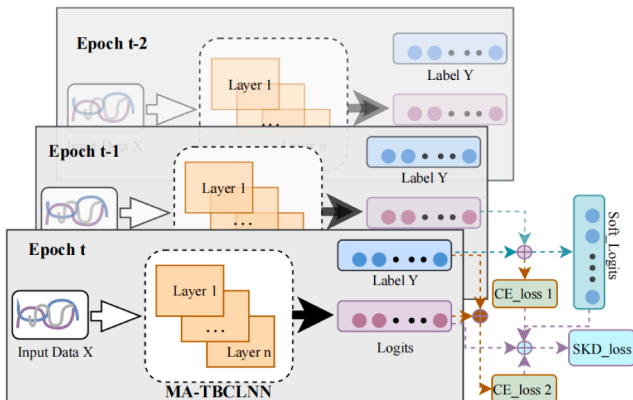


Fig. 3. Proposed APSKD method.

In knowledge distillation (KD), for an input x and a hard label y , the model produces a vector of predicted probability distributions $z(x)$ and then outputs the probabilities $P(x)$ via a Softmax function. To perform knowledge distillation even more, Hinton (2015) suggest using temperature moderation to soften these probabilities. Specifically, temperature conditioning smooths the probability distribution by introducing a temperature parameter T . The predicted probability $P(x)$ after temperature conditioning can be expressed as

$$P_i(x; \tau) = \frac{\exp(z_i(x)/\tau)}{\sum_j \exp(z_j(x)/\tau)} \quad (3)$$

where τ is the temperature parameter.

In APSKD, we use the model in epoch $(t-1)$ as the teacher and the model at epoch t as the student, so that the student model can benefit from the success and accumulation of knowledge of the teacher model in the previous epoch. In the epoch t training, we soften the hard label y and the predictive distribution probability $P_i^{t-1}(x)$ of the epoch $(t-1)$ to $(1-\alpha)y_i + \alpha P_i^{t-1}(x)$ as the soft label of epoch $(t-1)$. Among other things, in the early stages of training, the model often lacks sufficient experience to make accurate predictions. If it relies too much on its own prediction, it can easily lead to learning bias and unstable prediction results. And as training progresses, the model gradually accumulates a large amount of knowledge, and we need to encourage the model to trust its own previous predictions more to achieve self-reinforcement and internalization of knowledge. Therefore, we define α as a parameter that grows linearly with the number of training rounds, and its calculation formula is shown in Eq. 4.

$$\alpha_t = \alpha_T \times \frac{t}{T} \quad (4)$$

where T is the total number of epochs of training and α_T is the α of the last epoch.

Since an incorrect prediction of the epoch $(t-1)$ model may mislead another in knowledge transfer, we propose an adaptive method that weights the distillation loss according to the quality of the predicted soft labels. We first calculate the task loss for the epoch $(t-1)$ and epoch t models (denoted as L_i^{t-1} and L_i^t) using the standard cross-entropy loss function. The task loss is calculated as follows:

$$L_i^{t-1} = CE(P_i^{t-1}(x), y_i) \quad (5)$$

$$L_i^t = CE(P_i^t(x), y_i) \quad (6)$$

where $P_i^{t-1}(x)$ is the predicted probability of the i th sample in epoch $(t-1)$, and $P_i^t(x)$ is the predicted probability of the i th sample in epoch t .

Subsequently, the adaptive distillation loss function was used to calculate the distillation loss for the epoch $(t-1)$ and epoch t models as follows:

$$L_i^d = \frac{KL(P_i^t(x), P_i^{t-1}(x))}{L_i^{t-1} + L_i^t} \quad (7)$$

where KL is Kullback-Leibler Divergence, $KL(a, b) = -\sum_i a_i \log\left(\frac{b_i}{a_i}\right)$. In this case, if the predictions of the epoch $(t-1)$ model and the epoch t model are not reliable enough, i.e., they exhibit a large task loss, the weight or influence of knowledge distillation can be reduced. Conversely, if both the student model and the tutor model have been well trained to possess a smaller task loss, then the weight of knowledge distillation can be increased to make it a more important part of the total loss. Doing so would help to further optimize the student model by utilizing more accurate mentoring information, while possibly helping to reduce overfitting.

Therefore, the loss function updated by the model in epoch t in the APSKD training phase is shown in (8).

$$L_i^{APSKD} = L_i^t + L_i^d \quad (8)$$

Then, using the above loss function, the learning of the local model on the training set is updated by taking a gradient descent step,

$$\theta_i^1 = \theta_i^1 - \eta_i \nabla_{\theta_i^1} L_i^{APSKD} \quad (9)$$

where η_i is the learning rate of the local model of the i th client. We perform multiple iterations so that each training instance in an epoch can be used to update the local model. Model training under the guidance of APSKD is shown in Algorithm 1.

Algorithm 1 APSKD Training Algorithm.

Require: model M^0 , APSKD training epoch E , APSKD's learning rate η_k , Predicted distribution probabilities $P_i^{t-1}(x)$ for epoch $t-1$.
Ensure: model M^E .
1: **Producer:**
2: **for** $t \leq E$ **do**
3: **if** $t == 0$ **then**
4: The probability of the predicted distribution $P_i^{t-1}(x)$ at epoch $t-1$ is initialized to the tensor zero.
5: **end if**
6: Calculate the α_t by Eq. 4
7: Softening the $P_i^{t-1}(x)$ to $(1 - \alpha_t)y_i + \alpha_t P_i^{t-1}(x)$ and use it as $P_i^{t-1}(x)$
8: Compute prediction of probability distributions $P_i^t(x)$ by forward propagation
9: Calculation of APSKD loss L_i^{APSKD} by Eq. 5 - Eq. 8
10: Update the model parameters by Eq. 9
11: Use $P_i^t(x)$ as the new $P_i^{t-1}(x)$
12: **end for**
13: **return** model M^E .

3.3. Differential privacy based on uniform perturbation mechanisms

In traditional federated learning frameworks, multiple clients collaborate to train a shared model, periodically aggregating and updating the global model parameters. However, Shokri et al. (2017a) point out that the direct transmission of model parameters, while facilitating distributed learning, has significant privacy drawbacks. Even model parameters processed by differential privacy techniques can be recovered by malicious parties through “membership inference” or “attribute inference” attacks to reveal some features of the original data. To overcome these limitations, we use *Logits* of the predictive distribution probabilities of the transmission model, which significantly reduces the risk of information exposure while preserving the learning effect, making it extremely difficult for an attacker to recover the specifics of the model architecture or the training data even if the attacker is in possession of the model output.

Data security and privacy protection are crucial in IIoT environments, especially in application scenarios involving sensitive information, such as industrial control systems and production process monitoring. Therefore, protecting the privacy of users and devices and ensuring data security become essential to build trust and guarantee stable system operation. There are various methods for privacy protection, including data encryption, access control, and data anonymization. However, these methods often have certain limitations. For example, while data encryption can protect the security of data, it may increase the computational complexity when data is used and analyzed; while data anonymization may lead to the loss of information and affect the validity and usability of data. Differential privacy (DP), as an emerging privacy preservation approach, provides a powerful framework to balance the tension between data privacy and data usability. By adding random noise to the query results, differential privacy ensures that an attacker cannot accurately infer private information about other users even when certain data is known. The advantage is that differential privacy provides mathematically rigorous guarantees that enable users' privacy to be effectively protected during data analysis and machine learning.

To solve these problems, we introduce a differential privacy approach with the uniform perturbation mechanism (Geng et al., 2018).

The uniform perturbation mechanism is able to minimize the impact on the validity of the data while guaranteeing privacy protection by adding a uniformly distributed noise to the query results. This mechanism can better control the magnitude of the noise while realizing differential privacy, thus improving the accuracy and usefulness of the model.

The core concept of differential privacy preservation is to obfuscate the true contribution of individual data items by adding random noise to the query results independently of the original dataset to achieve effective masking of personal information. This query output independent noise addition mechanism is the basis of the standard approach to differential privacy. Specifically, assume that there is a dataset D , and when the query function is executed the query result $t = q(D)$ is obtained. To introduce privacy preservation, a mechanism K is used to add a random noise X from the probability distribution P to this result to form a noise-contaminated output:

$$K(D) = t + X \quad (10)$$

In this framework, the noise X usually follows some deterministic probability distribution, most typically the Laplace or Gaussian distributions, whose parameters are determined by the sensitivity of the query. If X follows the Laplace distribution, then the density function of X is:

$$f(x; b) = \frac{1}{2b} e^{-\frac{|x|}{b}} \quad (11)$$

where b is called the scale parameter, which determines the intensity of the noise and the shape of the distribution. In differential privacy scenarios, b is often associated with query sensitivity, ensuring that K satisfies the (ϵ, δ) -DP criterion, i.e., for any two datasets D_1 and D_2 differing by up to one record, and $\forall S \subseteq \text{Range}(K)$, there are

$$\Pr[K(D_1) \in S] \leq e^\epsilon \Pr[K(D_2) \in S] + \delta \quad (12)$$

where ϵ, δ are privacy parameters that quantify the degree of privacy protection, respectively, and $\epsilon, \delta > 0$. This suggests that even if an attacker has all other information except for a person's data, he or she is unable to significantly improve the ability to infer whether a person belongs to the dataset, thus safeguarding individual privacy.

To ensure that the differential privacy mechanism protects privacy while minimizing the impact on data quality and system performance, Geng et al. (2018) proposed the truncated Laplace mechanism. This mechanism decays as fast as possible and satisfies (ϵ, δ) -DP when the noise is small, and decays sharply to zero when the noise is large, to avoid generating costly heavy-tailed distributions. In particular, the truncated Laplace mechanism has a density function given the privacy parameters $0 < \delta < 1/2$, $\epsilon > 0$ and the query sensitivity $\Delta > 0$:

$$f_{TLap}(x) = \begin{cases} B e^{-\frac{x}{\lambda}}, & \text{for } x \in [-A, A] \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

$$\text{where } \lambda = \frac{\Delta}{\epsilon}, \quad A = \frac{\Delta}{\epsilon} \log \left(1 + \frac{\epsilon - 1}{2\delta} \right), \quad B = \frac{1}{2\lambda \left(1 - e^{-\frac{\Delta}{\lambda}} \right)}.$$

When performing differential privacy protection for predicted distribution probabilities that are in the range of $[0, 1]$ and are discrete values, the Laplace or Gaussian mechanism may require additional processing or cropping operations due to the possibility of exceeding the boundaries due to heavy-tailed distributions, while truncating the Laplace mechanism may cause data points near the boundaries to suffer disproportionately. For example, predicted values that were originally near 0 or 1 are forced to squeeze onto the boundary after adding noise and may no longer reflect the true probability estimates, a phenomenon known as the boundary effect, which can be detrimental to the accuracy and reliability of the prediction results. Therefore, we introduce the uniform perturbation mechanism, which is a special case of the truncated Laplace mechanism, where $\epsilon = 0$. Specifically, as $\epsilon \rightarrow 0$, the truncated Laplace distribution degenerates into a uniform distribution, and the probability density interval becomes $\left[-\frac{\Delta}{2\delta}, \frac{\Delta}{2\delta} \right]$.

The uniform perturbation mechanism has a uniform and consistent distribution of noise values over its defined domain. This means that

there is no spike near the center followed by a sharp decay as in the case of the Laplace or Gaussian mechanisms, and at the same time, it exhibits low boundary effects in practice, i.e., the query results in the edge regions are relatively less affected by the noise.

3.4. Global aggregation

Within the field of federated learning, the problem of non-independent homogeneous distributions is particularly prominent in the face of data heterogeneity and complexity, which makes the data owned by each participant show obvious imbalance in label distribution, feature space and even sample size. To quantify the unbalanced nature of the client-local dataset D_i^c , we use Shannon entropy to measure the unbalanced nature of each client-local dataset D_i^c . Specifically, the imbalance of a given dataset D containing k categories, each category corresponding to the number of samples c_i , and the total number of samples n is calculated as shown in Eq. 14.

$$\omega_{\text{entropy}} = \frac{-\sum_{i=1}^k \frac{c_i}{n} \log \frac{c_i}{n}}{\log k} \quad (14)$$

Among them, the larger Shannon entropy indicates the stronger internal diversity of the dataset, i.e., relatively more balanced between categories; in contrast, a small entropy value implies that a certain few or even a single category of samples dominates, resulting in an extremely unbalanced dataset.

However, Shannon entropy mainly focuses on the balance of category distribution within the local dataset D_i^c and ignores the cross-sectional comparison between datasets. This implies that the model may fall into a state of “myopia”, giving too much attention only to local high-frequency features and not sensitive to the performance in other categories or scenarios. Meanwhile, since we transmit the predicted distribution probability of the model in the public dataset D^p , if we simply use Shannon entropy as the aggregation weight, Shannon entropy can only reflect the balanced distribution of the categories in the local dataset D_i^c , but it can not directly reflect the existence of a consistent pattern or trend between these data and the public dataset D^p , which lacks the consistency consideration of the pattern among the datasets in the aggregation process consistency considerations. This would lead the model to overly focus on common features within the local dataset D_i^c , ignoring other features that may be equally important but less common in the local data, especially those represented in the public dataset D^p .

To address the above problem, we use maximum mean difference (MMD) (Borgwardt et al., 2006) to measure the difference between the local dataset D_i^c and the public dataset D^p based on the use of Shannon's entropy to characterize the distributions of the categories within the local dataset D_i^c . The core idea of MMD is to compare the expected embeddings of two random variables in a kernel Hilbert space, which provides a nonparametric way to test the similarity of two sample sets.

Definition of MMD: Assume that χ is the sample space and $k: \chi \times \chi \rightarrow \mathbb{R}$ denotes a positive definite kernel function of a feature mapping $k(x, y)$, and the corresponding Hilbert space is denoted as $\mathcal{H}_k$. When two sample datasets $X = x_1, x_2, \dots, x_m \in \chi^m$ and $Y = y_1, y_2, \dots, y_n \in \chi^n$ are given, the MMD is calculated as shown in Eq. 15.

$$\text{MMD}[F, X, Y] := \sup_{f \in F} \left(\frac{1}{m} \sum_{i=1}^m f(x_i) - \frac{1}{n} \sum_{j=1}^n f(y_j) \right) \quad (15)$$

where $\sup(\cdot)$ denotes the upper bound on the difference between the expectations of two datasets or instances, and $f(\cdot)$ is the radial basis function (RBF) kernel in $\mathcal{H}_k$. Here, we use the RBF of the Gaussian kernel function to compute the MMD distance.

$$K(x, y) = e^{-\frac{\|x-y\|^2}{2}} \quad (16)$$

The MMD distance calculation formula for each client's local dataset D_i^c and public dataset D^p is shown in Eq. 17.

$$\omega_{\text{MMD}} = \text{MMD}[F, X, Y] \quad (17)$$

Where X is the client's local dataset D_i^c , ($i = 1, 2, \dots, N$) and Y is the public dataset D^p .

Therefore, combining Shannon entropy and MMD to evaluate the complexity of category distribution within the local dataset D_i^c and the variability with the public dataset D^p , we propose entropy-maximum mean discrepancy (EMMD) to compute the weight ω_{EMMD} for the aggregation of the local *Logits*. Since EMMD focuses on the cross-domain data inter variability, it effectively reduces the possibility of overall model skewness caused by certain strong datasets dominating the learning process. Especially for data that contain extreme distributions or long-tailed categories, EMMD ensures that even a small but information-intensive amount of data receives the attention it deserves to adjust the model weights more accurately and mitigate the overfitting tendency of the federated model to adapt to diverse data characteristics. ω_{EMMD} is computed as shown in Eq. 18.

$$\omega_{\text{EMMD}} = \beta \times \omega_{\text{entropy}} + (1 - \beta) \times \omega_{\text{MMD}} \quad (18)$$

Subsequently, we globally aggregate the $\text{logits}_i^{\text{local}}$ from each client according to their respective aggregation weights to obtain the global $\text{logits}^{\text{global}}$, which is calculated as follows:

$$\text{logits}^{\text{global}} = \frac{\sum_{i=1}^N \text{logits}_i^{\text{local}} \times \omega_{\text{EMMD}}^i}{\sum_{i=1}^N \omega_{\text{EMMD}}^i} \quad (19)$$

3.5. Personalized knowledge distillation

PKD is an important component of the FTKD framework, which together with APSKD constitutes a two-stage knowledge distillation process to enhance the personalized learning capability of the local model. PKD is a technique designed to enhance the performance of each client's model by dynamically transferring knowledge in a federated learning framework to achieve more efficient model personalized learning. The core idea of the approach, shown in Fig. 4, is that each client utilizes the knowledge gained from the training of all clients on the local dataset D_i^c to guide the optimization process of the local model, thereby improving the model's adaptability and accuracy to specific attack scenarios.

In our proposed FTKD framework, the knowledge that passes the aggregation is the predicted probability distribution obtained by the model in the public training set, and in the PKD approach, we use the aggregated predicted probability distribution as the prediction of the teacher model to guide the learning of each client local model. Currently, in most of the knowledge distillation methods, knowledge-filling education is used, i.e., the teacher model directly instills knowledge to the student model, and the student model thus loses its autonomy in the learning process. To allow each client local model (student model) to learn what it needs in a more personalized way and to reduce knowledge forgetting, we introduced a proportional integral derivative (PID) control algorithm from the discipline of automation (Shao et al., 2021).

The PID control algorithm is a classical closed-loop control system algorithm applied in industrial automation, and its core idea is to automatically regulate the control output of the system by analyzing the

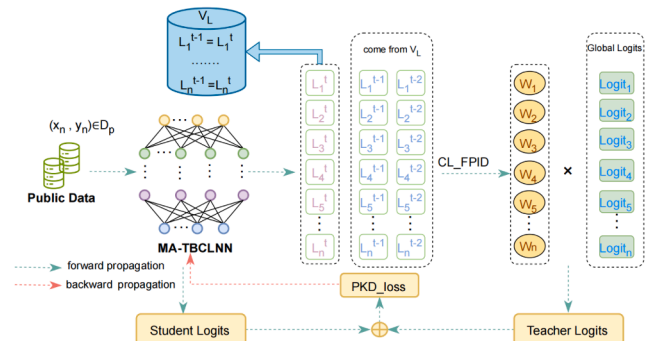


Fig. 4. Proposed PKD method.

proportional, integral, and differential components of the system deviation (the difference between the target value and the actual value) to accurately and stably control various physical processes. We apply it to knowledge distillation by dynamically adjusting the importance weights of each training sample according to its contribution to the overall performance, and multiplying the weights with the global *Logits* of the corresponding samples as the predictive probability distribution of the teacher model, thus forming a weighted predictive probability distribution of the teacher model, which makes the process of knowledge distillation of the local model more personalized.

The output $u(t)$ of the PID controller can be defined by Eq. 20.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (20)$$

where K_p, K_i, K_d are gain constants for proportional, integral, and differential, respectively. $e(t)$ is a unit of proportionality, which represents the error at moment t , i.e., the current loss of the local model on the samples in the public dataset D^p . $\int_0^t e(\tau) d\tau$ is the unit of integration of the cumulative error, i.e., the difficulty for students to acquire knowledge of the samples in the public dataset D^p . $\frac{de(t)}{dt}$ is the unit of derivative, which indicates the future trend of the error, i.e., the forgotten state of the students' knowledge about the samples in the validation set, and the predictive control by detecting the rate of change of the deviation (i.e., the derivative of the error). In PKD, $u(t)$ is used to control the learning weights of the aggregated *Logits* (i.e., the predictive probability distribution of the teacher model). If $u(t)$ is larger for a sample, then the teacher should pay more attention to the learning of that sample.

Yang et al. (2024) found that the derivative term responds more efficiently to the degree of knowledge forgotten when $n = 2$. Therefore, the traditional PID algorithm was improved as follows:

$$u(t) = K_p e(t), \quad t = 1 \quad (21)$$

$$u(t) = K_p e(t) + K_i \sigma \left(\frac{e(t) + e(t-1)}{2} \right) + K_d (e(t) + e(t-1)), \quad t = 2 \quad (22)$$

$$u(t) = K_p e(t) + K_i \sigma \left(\frac{e(t) + e(t-1) + e(t-2)}{3} \right) + K_d (e(t) + e(t-1) + e(t-2)), \quad t > 2 \quad (23)$$

where $e(t)$ is the loss L_i^t of sample i at moment t and $\sigma(\cdot)$ is the $\tanh(x)$ function.

However, setting the PID parameters correctly is the key to successfully using PID control; if they are just set to constants, there is no direct understanding of the process, resulting in an overall feedback algorithm that is not optimal. Therefore, we need to dynamically modify the three parameters of the PID control algorithm according to the learning state of the students to improve the PKD performance. To make the PID parameters automatically adapt to the learning progress of the local model, we introduce a fuzzy control strategy based on course learning (CL) (Yang et al., 2024) to adjust the three parameters of the PID control algorithm.

The CL-based fuzzy strategy dynamically adjusts the PID parameters according to the learning state (i.e., training loss) of the local model in the public dataset D^p . The strategy quantifies the average student performance on the most recent test, i.e., the average loss of all training samples in the $t-1$ st epoch, before determining the K_p, K_i , and K_d parameters for the current state. The learning state of the local model at the epoch $(t-1)$ can be formulated as:

$$\bar{e} = \frac{1}{N_p} \sum_{i=1}^{N_p} L_i^{t-1} \quad (24)$$

where N_p is the total number of training samples in the public dataset D^p and L_i^{t-1} is the loss value of the i th sample at epoch $(t-1)$.

Because a fuzzy set in the automation discipline usually consists of an affiliation function with seven linguistic variables (NB, NM, NS, ZO, PS, PM, PB), the strategy also classifies the range of $\bar{e}$ variation into seven classes. Specifically, if the maximum loss size of the local model on the public dataset D^p is L_m , and dividing L_m equally into an affiliation function m with seven linguistic variables, the affiliation function of L_m is as follows:

$$m_1 = \begin{cases} 1 & \text{if } \bar{e} < \frac{1}{7} L_m \\ \left(\frac{2}{7} L_m - \bar{e} \right) / \frac{1}{7} L_m & \text{if } \frac{1}{7} L_m \leq \bar{e} < \frac{2}{7} L_m \\ 0 & \text{otherwise} \end{cases} \quad (25)$$

$$m_{i \in \{2,3,4,5,6\}} = \begin{cases} \left(\bar{e} - \frac{i-1}{7} L_m \right) / \frac{1}{7} L_m & \text{if } \frac{i-1}{7} L_m \leq \bar{e} < \frac{i}{7} L_m \\ \left(\frac{i+1}{7} L_m - \bar{e} \right) / \frac{1}{7} L_m & \text{if } \frac{i}{7} L_m < \bar{e} \leq \frac{i+1}{7} L_m \\ 0 & \text{otherwise} \end{cases} \quad (26)$$

$$m_7 = \begin{cases} \left(\bar{e} - \frac{6}{7} L_m \right) / \frac{1}{7} L_m & \text{if } \frac{6}{7} L_m < \bar{e} \leq L_m \\ 1 & \text{if } \bar{e} > L_m \\ 0 & \text{otherwise} \end{cases} \quad (27)$$

$$m = [m_1, m_2, m_3, m_4, m_5, m_6, m_7] \in \mathbb{R}^7 \quad (28)$$

where the larger the value of m_1 , the better the performance of the local model; the higher the value of m_7 , the worse the performance of the local model.

At the same time, the proportional gain constant K_p is also divided into seven ranges of variation based on the same interval Δp , and the median value of K_p is taken as the initial value of K_p, K_{p0} :

$$K_p = [0, \Delta p, 2\Delta p, 3\Delta p, 4\Delta p, 5\Delta p, 6\Delta p] \in \mathbb{R}^7, K_{p0} = 3\Delta p \quad (29)$$

where Δp is a hyperparameter that controls the range of values of the proportional gain constant K_p .

Next, defuzzification is performed and the center of mass is used to derive the exact K_p value:

$$K_p = \sum_{i=0}^7 K_{pi} \times m_i \quad (30)$$

Therefore, the integral gain constant K_i and the differential gain constant K_d are calculated as follows:

$$K_i = K_{i0} + (K_{p0} - K_p) \quad (31)$$

$$K_d = K_{d0} + (K_{p0} - K_p) \quad (32)$$

where K_{i0} and K_{d0} are hyperparameters and both K_i and K_d are greater than or equal to zero.

By dynamically adjusting the three gain parameters of the PID algorithm through the CL-based fuzzy strategy, we can get the feedback value $u(t)$ of the PID algorithm, and then use the softmax function to calculate the importance of the sample x_i^p in the public dataset D^p to the current local model:

$$w_i = \text{softmax}(u_i(t)) = \frac{\exp(u_i(t))}{\sum_{i=1}^{N_p} \exp(u_i(t))} \quad (33)$$

After obtaining the weights w_i of the samples x_i^p in each client's public dataset D^p , we multiply the weights w_i with the global *Logits* and use them as the predictive probability distribution *Logits* of the teacher model for instructive training of the local model. In this case, the predictive probability distribution of the teacher model is calculated as shown in Eq. 34:

$$P_i^T(x) = \text{logits}^{\text{global}} \times w_i \quad (34)$$

Since all industrial IoT data are imbalance class data and to better improve the generalization and robustness of the local model, we use focus loss (Ross & Dollár, 2017) as the task loss in PKD. First, we calculate the local model training loss and the prediction loss of the teacher model using the focal loss function,

$$L_{FL}^S = (1 - P_i^S(x))^r \sum_{N_P} -y_i \log(P_i^S(x)) \quad (35)$$

$$L_{FL}^T = (1 - P_i^T(x))^r \sum_{N_P} -y_i \log(P_i^T(x)) \quad (36)$$

where $P_i^S(x)$ and $P_i^T(x)$ are the predicted values of the local model and the teacher model, respectively, for sample i , and y_i is the true value for sample i .

Second, we use KL scatter to compute the adaptive distillation loss for the local and teacher models, which is calculated as follows:

$$L_{KL} = \frac{KL(P_i^S(x), P_i^T(x))}{L_{FL}^S + L_{FL}^T} \quad (37)$$

In this case, distillation strength is weaker if the predictions of the local and teacher models are unreliable, i.e., they have large task losses. If the local and teacher models were well tuned for small task losses, distillation losses became dominant, which acted as a regularizing effect, potentially mitigating the risk of overfitting.

Ultimately, the loss function calculation formulation for PKD is shown in Eq. 38:

$$L_{PKD} = L_{FL}^S + L_{KL} \quad (38)$$

Then, using the above loss function, the learning of the local model on the training set is updated by taking a gradient descent step,

$$\theta_i^l = \theta_i^l - \nu_i \nabla_{\theta_i^l} L_i^{PKD} \quad (39)$$

where ν_i is the learning rate of the local model of the i -th client. We perform multiple iterations so that each training instance in an epoch can be used to update the local model. PKD-guided model training is shown in Algorithm 2.

Algorithm 2 PKD Training Algorithm.

Require: model M^{r-1} , PKD's learning rate ν_k , Current number of rounds r , Loss of each sample of the public dataset L_i^{r-1} at round $r-1$, Loss of each sample of the public dataset L_i^{r-2} at round $r-2$.

Ensure: model M^r , L_i^{r-1} and L_i^{r-2} .

- 1: Calculate weights w_i for each sample in the public dataset D^p based on L_i^{r-1} and L_i^{r-2} by Eqs. 21–33
 - 2: Calculate the probability distribution $P_i^T(x)$ of teacher model predictions by Eq. 34
 - 3: Calculate of PKD loss L_i^{PKD} by Eqs. 35–38
 - 4: Update the model parameters by Eq. 39
 - 5: Set L_i^r as the new L_i^{r-1} and L_i^{r-1} as the new L_i^{r-2}
 - 6: **Return** model M^r , L_i^{r-1} and L_i^{r-2}
-

Algorithm 3 outlines the complete model workflow in FL training processes. Throughout the iterative FL process, the IoT client interacts continuously with the FL server to update the local model parameters and refine the intrusion detection model through multiple rounds of communication.

4. Convergence analysis of the FTKD method

4.1. Convergence analysis of the APSKD loss function

Definition of the Loss Function:

$$L_t^{APSKD} = CE(P_t^l(x), y) + \frac{KL(P_t^l(x), P_{t-1}^l(x))}{L_t^l + L_{t-1}^l} \quad (40)$$

Algorithm 3 FTKD Framework Workflow.

Require: Clients set $C = \{c_1, c_2, \dots, c_k\}$, Number of communication rounds R , APSKD training epoch E , APSKD's learning rate η_k , PKD's learning rate ν_k .

Ensure: Personalized models $\{M_i^R\}_{i \in K}$.

- 1: Initialization: Initial model parameters;
 - 2: **for** $r = 1$ to R **do**
 - 3: (1) For Clients:
 - 4: **for all** $i \in k$ **do**
 - 5: Client c_i calculates the local aggregation weight $\omega_{EMMD}^{c_i}$ by Eqs. 14–18;
 - 6: Client c_i updates the model parameters $w_{c_i}^r$ using Algorithm 1 and selects the best model M_{c_i} ;
 - 7: Use the best model $M_{c_i}^r$ to predict the public dataset D_p to get local predictions $\text{logits}_{c_i}^{local}$ and record the loss $L_{c_i}^r$;
 - 8: **end for**
 - 9: DP protection for $\text{logits}_{c_i}^{local}$ using the uniform perturbation mechanism;
 - 10: Upload $\omega_{EMMD}^{c_i}$ and $\text{logits}_{c_i}^{local}$ to Server;
 - 11: (2) For Server:
 - 12: Calculate the global prediction logits^{global} by Eq. 19;
 - 13: Send logits^{global} to Clients;
 - 14: (1) For Clients:
 - 15: **for all** $i \in k$ **do**
 - 16: Client c_i updates the model parameters $w_{c_i}^r$ using Algorithm 2;
 - 17: **end for**
 - 18: **end for**
 - 19: **return** personalized models $\{M_i^R\}_{i \in K}$.
-

Step 1: Gradient Decomposition and Boundedness

The gradient with respect to parameters θ_t is given by:

$$\nabla L_t^{APSKD} = \nabla CE(P_t^l, y) + \nabla \left(\frac{KL(P_t^l, P_{t-1}^l)}{L_t^l + L_{t-1}^l} \right) \quad (41)$$

The second term can be expanded as:

$$\nabla \left(\frac{KL}{L_t + L_{t-1}} \right) = \frac{\nabla KL \cdot (L_t + L_{t-1}) - KL \cdot (\nabla L_t + \nabla L_{t-1})}{(L_t + L_{t-1})^2} \quad (42)$$

where KL denotes the abbreviation of $KL(P_t^l, P_{t-1}^l)$. Under the assumptions that $KL \leq K$ and $L_t + L_{t-1} \geq L_{\min} > 0$, there exist constants C_1 and C_2 such that:

$$\left\| \nabla \left(\frac{KL}{L_t + L_{t-1}} \right) \right\| \leq \frac{C_1}{L_{\min}} + \frac{C_2 K}{L_{\min}^2} \quad (43)$$

Consequently, the total gradient satisfies:

$$\|\nabla L_t^{APSKD}\| \leq G + \frac{C_1}{L_{\min}} + \frac{C_2 K}{L_{\min}^2} \triangleq C \quad (44)$$

where G represents the global upper bound of the gradient $\nabla CE(P_t^l, y)$. Thus, the gradient ∇L_t^{APSKD} is globally bounded.

Step 2: Weak Convexity Analysis of the APSKD Loss Function

Definition 1 (Weak Convexity): A function $L(\theta)$ is said to be weakly convex if there exists a constant $\rho > 0$ such that for any parameters θ, θ' , the following inequality holds:

$$L(\theta') \geq L(\theta) + \nabla L(\theta)^T (\theta' - \theta) - \frac{\rho}{2} \|\theta' - \theta\|^2 \quad (45)$$

In multi-class classification tasks, the cross-entropy loss is convex, and the KL-divergence is also inherently convex. However, the dynamic nonlinearity introduced by the denominator $L_t + L_{t-1}$ may compromise global convexity. To address this, we decompose the APSKD loss into local and non-local components. Specifically, $CE(P_t(x), y)$ constitutes the local component, while $KL(P_t(x), P_{t-1}(x))/(L_t + L_{t-1})$ represents the non-local component.

To rigorously establish the weak convexity of the APSKD loss, we must demonstrate that the influence of the non-local component on overall weak convexity remains controllable. In Step 1, we have already proven the boundedness of the non-local component.

For the non-local component $L_{\text{non-local}}(\theta)$, consider its second-order Taylor expansion at θ :

$$L_{\text{non-local}}(\theta') = L_{\text{non-local}}(\theta) + \nabla L_{\text{non-local}}(\theta)^T (\theta' - \theta) + \frac{1}{2} (\theta' - \theta)^T \nabla^2 L_{\text{non-local}}(\xi) (\theta' - \theta) \quad (46)$$

where ξ lies between θ and θ' . According to the definition of weak convexity, the following must hold:

$$\frac{1}{2} (\theta' - \theta)^T \nabla^2 L_{\text{non-local}}(\xi) (\theta' - \theta) \geq -\frac{\rho_{\text{non-local}}}{2} \|\theta' - \theta\|^2 \quad (47)$$

This implies the requirement:

$$\nabla^2 L_{\text{non-local}}(\xi) \geq -\rho_{\text{non-local}} I \quad (48)$$

where $\nabla^2 L_{\text{non-local}}$ can be decomposed as:

$$\nabla^2 L_{\text{non-local}} = \frac{\nabla^2 \text{KL}}{L_t + L_{t-1}^t} - \frac{\text{KL} \cdot \nabla^2 (L_t + L_{t-1})}{(L_t + L_{t-1}^t)^2} \quad (49)$$

Since the KL-divergence is convex, $\nabla^2 \text{KL} \geq 0$. Under the assumption $L_t + L_{t-1} \geq L_{\min} > 0$, the upper bound of the second term in Eq. 49 is given by:

$$\left\| \frac{\text{KL} \cdot \nabla^2 (L_t + L_{t-1})}{(L_{\min})^2} \right\| \leq \frac{K \cdot G''}{L_{\min}} \quad (50)$$

Therefore, there exists a constant $\rho = \frac{K \cdot G''}{L_{\min}}$ such that $\nabla^2 L_{\text{non-local}}(\xi) \geq -\rho_{\text{non-local}} I$. This confirms that the APSKD loss function is weakly convex.

Step 3: Parameter Update and Convergence Analysis

The parameter update rule is given by:

$$\theta_{t+1} = \theta_t - \eta_t \nabla L_t^{\text{APSKD}} \quad (51)$$

Based on the stochastic gradient descent (SGD) convergence theorem (Bottou, 2010), and given the global boundedness of the gradient $\nabla L_t^{\text{APSKD}}$ (established in Step 1), if the learning rate η_t satisfies:

$$\sum_{t=1}^{\infty} \eta_t = \infty, \quad \sum_{t=1}^{\infty} \eta_t^2 < \infty \quad (52)$$

then the parameter sequence $\{\theta_t\}$ guarantees:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E} [\|\nabla L(\theta_t)\|^2] = 0 \quad (53)$$

Furthermore, by the convergence theorem for weakly convex functions (Duchi et al., 2011), the sequence $\{\theta_t\}$ converges almost surely to a stationary point:

$$\lim_{t \rightarrow \infty} \mathbb{E} [\|\nabla L(\theta_t)\|] = 0 \quad (54)$$

4.2. Convergence analysis of the PKD loss function

Definition of the Loss Function:

$$L_{\text{PKD}} = (1 - P_i^S(x))^\gamma \sum -y_i \log(P_i^S(x)) + \frac{\text{KL}(P_i^S(x), P_i^T(x))}{L_{\text{FL}}^S + L_{\text{FL}}^T} \quad (55)$$

Step 1: Strong Convexity Analysis of Focal Loss

The focal loss is expressed as:

$$L_{\text{FL}}^S = - \sum y_i (1 - P_i^S)^\gamma \log(P_i^S) \quad (56)$$

The second derivative (Hessian matrix) with respect to P_i^S ,

$$\frac{\partial^2 L_{\text{FL}}^S}{\partial P_i^S \partial P_j^S} = \begin{cases} \frac{y_i (1 - P_i^S)^{\gamma-1} (\gamma + (1 - P_i^S))}{(P_i^S)^2}, & i = j \\ 0, & i \neq j \end{cases} \quad (57)$$

Since $P_i^S \in (0, 1)$ and $\gamma \geq 1$, the Hessian matrix is positive definite, indicating that the loss function satisfies strong convexity in local regions. Consequently, gradient descent converges at a linear rate:

$$\|\theta_{t+1} - \theta^*\| \leq \kappa \|\theta_t - \theta^*\|, \quad \kappa \in (0, 1) \quad (58)$$

Step 2: Stability Proof of PID Control

Define the error $e(t) = L_{\text{FL}}^S(t) - L_{\text{FL}}^T(t)$. The PID controller output is given by:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (59)$$

Construct the Lyapunov function:

$$V(t) = \frac{1}{2} e(t)^2 + \frac{K_i}{2} \left(\int_0^t e(\tau) d\tau \right)^2 + \frac{K_d}{2} \left(\frac{de(t)}{dt} \right)^2 \quad (60)$$

Differentiate and substitute the PID equation:

$$V'(t) = e(t)e'(t) + K_i \int_0^t e(\tau) d\tau \cdot e(t) + K_d \frac{de(t)}{dt} \cdot \frac{d^2 e(t)}{dt^2} \quad (61)$$

This work ensures $V'(t) \leq 0$ by configuring $K_p, K_i, K_d > 0$, which guarantees exponential convergence of the error $e(t)$ to zero.

Step 3: Global Convergence Analysis of PKD Loss Function

The PID controller ensures smooth variation of weights w_t , aligning the gradient directions of focal loss and KL-divergence. The parameter update rule is given by:

$$\theta_{t+1} = \theta_t - \eta_t (\nabla L_{\text{FL}}^S + w_t \nabla L_{\text{KL}}) \quad (62)$$

Given the strong convexity of focal loss, bounded gradients of KL-divergence, and PID-controlled stability, the PKD update gradients satisfy:

$$\|\nabla L_{\text{PKD}}\| \leq \|\nabla L_{\text{FL}}^S\| + w_t \|\nabla L_{\text{KL}}\| \leq G + B \quad (63)$$

According to stochastic gradient descent convergence conditions, the parameter sequence $\{\theta_t\}$ guarantees:

$$\lim_{t \rightarrow \infty} \mathbb{E} [\|\nabla L_{\text{PKD}}(\theta_t)\|] = 0 \quad (64)$$

This proves that the PKD loss function converges to a stationary point.

5. Experiment setup

The federated learning framework construction, model training, and model testing in this experiment were performed on a laptop with an Intel Core i5-9300H, NVIDIA GeForce GTX 1650 GDDR6, and 32GB of RAM. It is worth noting that during the client process, all clients are synchronized using multi-process parallelism to train the local model as well as to predict on the public dataset.

5.1. Datasets

This experiment provides a comprehensive evaluation of the proposed FTKD method on ToN-IoT (Moustafa, 2021) and Edge-IIoTset (Ferrag et al., 2022) intrusion detection datasets.

The ToN-IoT dataset is composed of heterogeneous data sources collected from telemetry data of IIoT services, as well as operating system logs and network traffic of IIoT networks, which include nine types of cyber-attacks namely Scanning, DoS, DDoS, Ransomware, Backdoor, Injection, XSS, Password and MITM. The Train_Test_Network dataset in the file "Train_Test_datasets" is selected for this experiment, which contains a total of 461,043 network traffic samples and 43 features.

The Edge-IIoTset dataset identifies and analyzes 14 attacks related to IIoT and IIoT connectivity protocols. The dataset contains a total of 2,219,201 network traffic samples with no missing values but 815 duplicates. We apply one-hot encoding to some features, and the dataset after preprocessing contains 2,218,386 network traffic samples and 110

Table 1
Training and test distributions.

Dataset	Configuration	Percentage of each type	Total	W_{EMMD}
ToN-IoT	Public(D^P)	Nomal(65 %), Backdoor(4.3 %), DDoS(4.3 %), DoS(4.3 %), Injection(4.3 %), Password(4.3 %), Ransomware(4.3 %), Scanning(4.3 %), XSS(4.3 %), Mitm(0.6 %)	23,053	–
	Client1(D_1^C)	Nomal(81.3 %), Backdoor(0.3 %), DDoS(2.3 %), DoS(2.4 %), Injection(13.7 %)	89,442	0.2588
	Client2(D_2^C)	Nomal(16.3 %), Backdoor(6 %), DDoS(4.6 %), DoS(0.5 %), Injection(3.3 %), Password(7.7 %), Ransomware(20.2 %), Scanning(20.1 %), XSS(20.2 %), Mitm(1.1 %)	75,340	1.2076
	Client3(D_3^C)	Nomal(80 %), Backdoor(0.3 %), DDoS(5.1 %), DoS(4.4 %), Injection(0.5 %), Password(9.7 %)	95,872	0.2759
	Client4(D_4^C)	Nomal(74 %), Backdoor(11.3 %), DDoS(5.3 %), DoS(9.4 %)	89,738	0.3052
	Test(D_{test})	Nomal(64.8 %), Backdoor(4.5 %), DDoS(4.4 %), DoS(4.4 %), Injection(4.4 %), Password(4.3 %), Ransomware(4.3 %), Scanning(4.3 %), XSS(4.3 %), Mitm(0.3 %)	87,598	–
Edge-IIoTset	Public(D^P)	Nomal(24.9 %), DDoS_UDP(15.1 %), DDoS_ICMP(14.5 %), SQL_injection(6.4 %), Password(6.2 %), Vulnerability_scanner(6.2 %), DDoS_TCP(6.2 %), DDoS_HTTP(6.2 %), Uploading(4.7 %), Backdoor(3.1 %), Port_Scanning(2.8 %), XSS(2 %), Ransomware(1.4 %), Fingerprinting(0.2 %), Mitm(0.1 %)	40,138	–
	Client1(D_1^C)	Nomal(22.7 %), DDoS_UDP(0.2 %), DDoS_ICMP(0.3 %), SQL_injection(2.8 %), Password(0.6 %), Vulnerability_scanner(8.7 %), DDoS_TCP(23.3 %), DDoS_HTTP(22.9 %), Uploading(18.5 %)	156,166	0.7787
	Client2(D_2^C)	Nomal(2.9 %), DDoS_UDP(37.8 %), DDoS_ICMP(41.9 %), SQL_injection(17.4 %)	173,477	0.9106
	Client3(D_3^C)	Nomal(8.4 %), DDoS_UDP(8.1 %), DDoS_ICMP(2 %), SQL_injection(3.2 %), Password(19 %), Vulnerability_scanner(9.9 %), DDoS_TCP(1.3 %), DDoS_HTTP(1.7 %), Backdoor(15.2 %), Port_Scanning(13.9 %), XSS(9.7 %), Ransomware(6.7 %), Fingerprinting(0.6 %), Mitm(0.3 %)	123,921	0.8868
	Client4(D_4^C)	Nomal(64.8 %), DDoS_UDP(10.5 %), DDoS_ICMP(8.1 %), SQL_injection(0.2 %), Password(8.7 %), Vulnerability_scanner(7.7 %)	156,450	0.4119
	Test(D_{test})	Nomal(24.8 %), DDoS_UDP(15.2 %), DDoS_ICMP(14.5 %), SQL_injection(6.4 %), Password(6.2 %), Vulnerability_scanner(6.2 %), DDoS_TCP(6.2 %), DDoS_HTTP(6.2 %), Uploading(4.7 %), Backdoor(3.1 %), Port_Scanning(2.8 %), XSS(2 %), Ransomware(1.4 %), Fingerprinting(0.2 %), Mitm(0.1 %)	152,521	–

features. Due to the limited hardware resources for the experiment, we performed undersampling on the normal traffic and randomly selected 200,000 of these samples, and the final dataset contains 802,743 traffic samples and 110 features.

Table 1 presents the distribution of training data from both the ToN-IoT dataset and the Edge-IIoTset dataset across the public dataset D^P and four client datasets. We classify 5 % of the dataset as the public dataset, 80 % of the remaining dataset as the training set, and 20 % as the test set. Meanwhile, we set up 4 different class imbalance and non-IID datasets in the clients. The test set is co-distributed with the original dataset, allowing the evaluation metrics to better reflect the performance of the proposed method in the real world.

5.2. Evaluation settings

In our experiments, we use a set of different key metrics to evaluate the performance of intrusion detection models. These metrics include Accuracy, Recall, and F1-Score, which are defined by the following formulas:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (65)$$

$$\text{Recall} = \frac{TP}{FN + TP} \quad (66)$$

$$F_1 = \frac{2TP}{2TP + FP + FN} \quad (67)$$

where TP denotes the number of positive samples correctly predicted as positive; TN is the number of negative samples correctly predicted as negative; FP is the number of negative samples incorrectly predicted as positive; and FN is the number of positive samples incorrectly predicted as negative.

In addition, when evaluating computational costs, we typically refer to several key metrics, including FLOPs and the number of parameters in the model.

6. Experiment results

This section presents the experimental results of the FTKD method in four main parts. The first part is the multi-classification performance results of the FTKD method for each client traffic flow. The second part

Table 2
Performance result for each client.

Dataset	Client	Accuracy (%)	Recall (%)	F1-Score (%)
ToN-IoT	Client1	97.29	97.29	97.16
	Client2	98.38	98.38	98.39
	Client3	97.73	97.73	97.61
	Client4	97.28	97.28	97.25
Edge-IIoTset	Client1	97.87	97.87	97.34
	Client2	99.02	99.02	99.01
	Client3	99.49	99.49	99.48
	Client4	97.47	97.47	97.35

is a comparison experiment with existing federated learning methods as well as centralized intrusion detection methods. The third part is to conduct various ablation studies and perform thorough qualitative evaluation to effectively highlight the proposed FTKD method. In the fourth part to emphasize the superiority of the FTKD method in terms of privacy preservation, we analyze its privacy properties.

6.1. Performance results for localized model

Table 2 shows the performance evaluation of each client on the FTKD method. The results show that the FTKD method performs very well on both the ToN-IoT dataset and the Edge-IIoTset dataset, and the accuracy, recall, and F1-score of each client are above 97 %. It is worth noting that the performance of each client is closely related to the local data distribution, and our proposed EMD method can effectively improve the accuracy of clients with few or unbalanced data categories. This demonstrates the effectiveness of the FTKD approach and its potential to facilitate cross-client knowledge transfer.

To more intuitively show the classification effect of the FTKD method on each client, we use T-SNE to visualize the classification effect of the method, as shown in Fig. 5. This figure uses the ToN-IoT test set with 20,000 samples to show the data distribution of each client before and after classification. Fig. 5(a) shows the traffic distribution before classification, and the distributions of various types of traffic appear to be haphazard and difficult to distinguish. However, in Fig. 5(b)–(e), after being processed via the FTKD method, each type of traffic forms relatively concentrated clusters, allowing different types of traffic to be clearly distinguished. Notably, Client1 is not only able to effectively

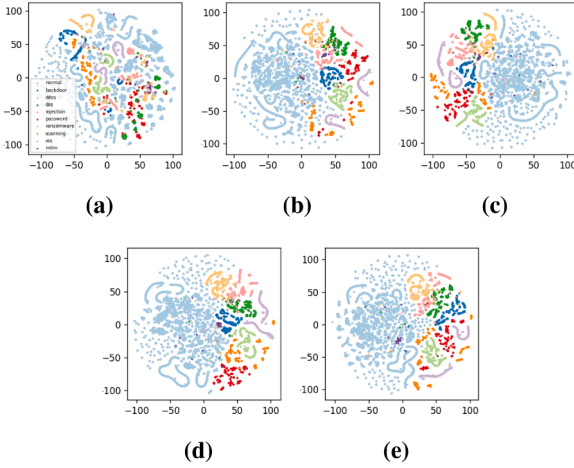


Fig. 5. Visualization of FTKD performance on the ToN-IoT dataset via T-SNE. (a) Original data. (b) Data processed via the FTKD method in Client 1. (c) Data processed via the FTKD method in Client 2. (d) Data processed via the FTKD method in Client 3. (e) Data processed via the FTKD method in Client 4.

cluster the existing attack traffic (Normal, Backdoor, DDoS, etc.) in the local dataset but also able to group the unseen attack traffic (e.g., Password, Ransomware, etc.) into one category, which indicates that the FTKD approach improves the generalizability of each client, which enables it to effectively recognize new types of attacks. In addition, each client shows greater recognition of its own specific attack traffic than others do, which further demonstrates that the FTKD method can promote the personalization of the local model and enhance its ability to recognize specific attack patterns.

6.2. Ablation studies

This subsection investigates the effect of hyperparameters on student performance and assesses the validity of each module of the proposed methodology.

1) Impact of hyperparameters on the FTKD methodology

In this paper, the hyperparameters Δp , K_{i0} and K_{d0} determine K_p , K_i and K_d for controlling the size of the proportional, integral and derivative units in the improvement algorithms, respectively. L_m is the magnitude of the cross-entropy loss of the samples in the public dataset D^p . The number of linguistic variables included in the affiliation function m determines the granularity of the student performance assessment system. τ is the distillation temperature, which is used to control the smoothness of knowledge transfer in APSKD and PKD. β is used to balance the weights between Shannon's entropy and MMD to provide a more reasonable aggregation weight based on the information about the distribution of the traffic categories in the local dataset and the public dataset.

We choose the ToN-IoT dataset and Edge-IIoTset dataset in the traffic flow multategorization task to verify the effects of these hyperparameters on student performance. The experimental results show that the ToN-IoT dataset has the best performance for multi-categorization task detection under the following hyperparameters: $\Delta p = 0.06$, $K_{i0} = 0.06$, $K_{d0} = 0.1$, $L_m = 2.1$, $\tau = 0.2$, $\beta = 0.5$, and 7 linguistic variables. The Edge-IIoTset dataset has the best performance for multi-categorization task detection when $\Delta p = 0.04$, $K_{i0} = 0.04$, $K_{d0} = 0.08$, $L_m = 2.2$, $\tau = 0.1$, $\beta = 0.5$ and 7 linguistic variables.

2) Impact of the proposed FTKD method on detection accuracy

To further understand the importance of each component of the FTKD approach, we performed several ablation studies as described below:

- MA-TBCLNN-FedAvg: MA-TBCLNN model using the FedAvg framework as a baseline for ablation studies.

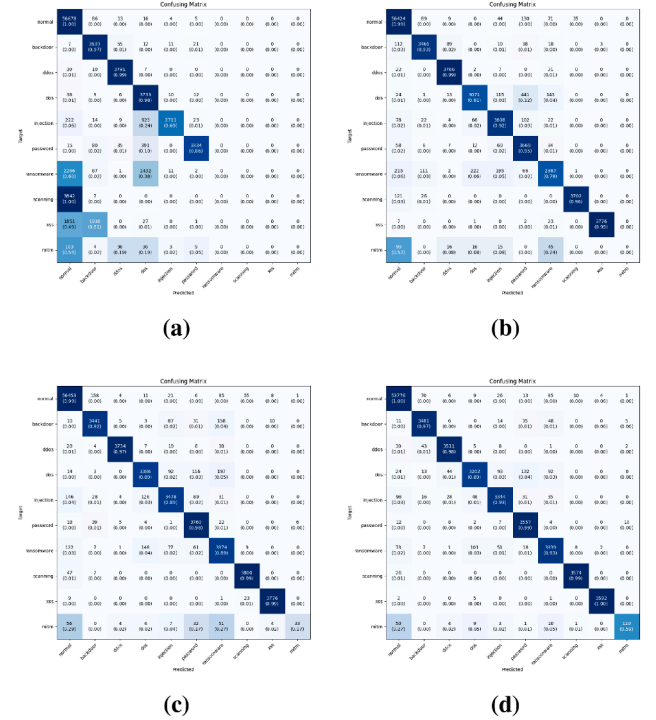


Fig. 6. Confusion matrix for FTKD method ablation studies on Client 1. (a) MA-TBCLNN-FedAvg. (b) MA-TBCLNN-PKD. (c) MA-TBCLNN-PKD-EMMD. (d) MA-TBCLNN-FTKD.

- MA-TBCLNN-PKD: the MA-TBCLNN model uses a multiclassification representation of the PKD method, where Logits aggregation uses average aggregation.
- MA-TBCLNN-PKD-EMMD: multiclassification performance of the MA-TBCLNN model with the PKD method as well as the EMD method.
- MA-TBCLNN-FTKD(Our): multiclassification performance of MA-TBCLNN model in our proposed FTKD method.
- TBCLNN-FTKD: multiclassification performance of the TBCLNN model in our proposed FTKD approach.

The experimental results are shown in Table 3. The experimental results reveal that our proposed PKD method, EMD aggregation method, and APSKD method are all conducive to improving the accuracy and generalization ability of each client. To explore the specific roles of these methods in the multi-classification task in more depth, we further employ the confusion matrix for visualization and analysis, as shown in Fig. 6.

In the ablation study in this paper, we mainly show the multiclassification results of Client1. Fig. 6(a) demonstrates the multi-classification results of the MA-TBCLNN model in the FedAvg framework, which is used as the base line for the ablation study. From Fig. 6(a), the detection accuracies of the four categories of traffic, Normal, Backdoor, DDoS, and DoS, are all over 97%, while the detection accuracy of the remaining six categories of attacks (Injection, Password, Ransomware, etc.) are very low, especially for Ransomware, Scanning, XSS, and MitM, the detection accuracy is 0. This phenomenon occurs because Client1's local dataset contains only the first four types of traffic samples, while the last six types do not appear in the training set, and thus the model can only effectively identify the types of traffic it has been trained on. This phenomenon also reveals the limitations of the FedAvg method in enhancing the generalization ability of each client.

Fig. 6(b) shows the multi-classification results using the PKD method, whose global aggregation is through an average aggregation strategy. The results show that the detection accuracies of the last six attack types

Table 3
Ablation studies of FTKD method.

Work	ToN-IoT			Edge-IIoTset		
	Accuracy (%)	Recall (%)	F1-Score (%)	Accuracy (%)	Recall (%)	F1-Score (%)
MA-TBCLNN-FedAvg	87.61	87.51	85.34	86.54	86.49	86.47
MA-TBCLNN-PKD	96.35	96.35	96.29	96.24	96.24	96.12
MA-TBCLNN-PKD-EMMD	97.16	97.16	97.07	96.93	96.93	96.78
TBCLNN-FTKD	97.28	97.28	97.11	97.17	97.17	97.03
MA-TBCLNN-FTKD (Ours)	97.67	97.67	97.60	98.46	98.46	98.29

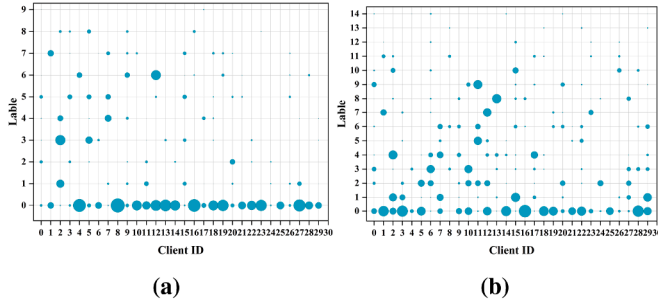


Fig. 7. The data distribution on the two datasets. (a) ToN-IoT. (b) Edge-IIoTset.

are improved to varying degrees, which is since PKD allows the model to learn the detection experience of other clients for the unseen attack types through the knowledge distillation mechanism. Fig. 6(c) introduces the EMMD policy for global aggregation based on PKD. Compared to Fig. 6(b), Fig. 6(c) further improves the detection accuracy of the last six types of traffic, reaching more than 88 % for all types except very few MitM traffic. Meanwhile, the detection accuracy of MitM attack traffic, which originally had an accuracy of 0, also improved to 33 %. However, this generalized learning through knowledge distillation also leads to a slight decrease in the detection accuracy of the first four types of traffic, e.g., the accuracy of DoS attack traffic is reduced from 98 % to 89 %. Therefore, to address the above issues and further enhance the learning of local dataset knowledge, we introduce APSKD in Fig. 6(d). Experimental results show that the detection accuracy of all types of traffic is improved compared to that of Fig. 6(c) with the addition of APSKD, thus achieving a more balanced and efficient detection of all types of traffic flows.

6.3. Performance results for comparison studies

The first two sections have already conducted in-depth algorithm performance analysis on four clients and carefully observed their personalized learning and convergence characteristics. To further systematically evaluate the scalability and robustness of the FTKD algorithm in large-scale, highly heterogeneous scenarios, this study will expand the number of clients to 30 to further validate the effectiveness of the FTKD algorithm. Fig. 7 shows the data distribution of each client under different datasets.

1) Comparison with state-of-the-art FL methods

We conduct a comparison experiment between the proposed FTKD and the current mainstream non-IID solutions. As a well-known algorithm in the federated learning field, FedAvg (McMahan et al., 2017) is widely used in many IIoT intrusion detection schemes. In addition, for the non-IID problem, Li et al. (2020) proposed FedProx, a federated learning strategy. This method effectively mitigates the client-side model drift problem caused by non-IID data distribution by introducing regularization terms in the local model update. Meanwhile, Li et al. (2021) also proposed a new method called FedBN, which improves the performance of model training in a federated learning environment by mitigating the bias of the feature distribution before model averaging

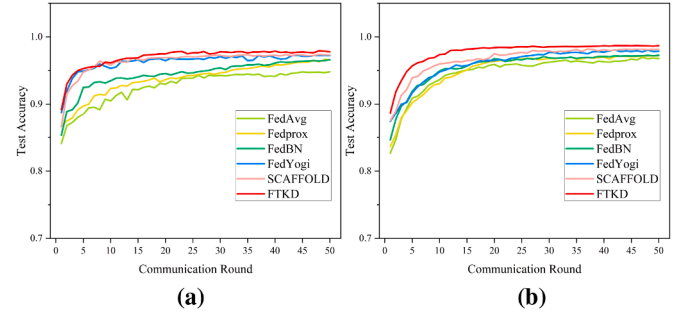


Fig. 8. Trends in accuracy of different methods on the two datasets. (a) ToN-IoT. (b) Edge-IIoTset.

using local batch normalization. It is worth mentioning that, to address the issue that FedAvg may exhibit unstable or slow convergence under significant data heterogeneity, Reddi et al. (2020) proposed FedYogi, which employs the Yogi adaptive optimizer on the server side to improve convergence speed and robustness by dynamically adjusting the global learning rate. Meanwhile, Karimireddy et al. (2020) introduced the SCAFFOLD algorithm, which utilizes control variables on both the client and server sides to correct local gradient bias, thereby directly mitigating client drift.

We have computed the performance metrics of the different methods and the time overhead required for each round of training and communication, as shown in Table 4. When facing the challenges posed by heterogeneous data distributions, the traditional FedAvg method has the worst performance due to its lack of effective adaptation to diversity and local variability, while FedProx, FedBN, FedYogi and SCAFFOLD have intermediate performance. Our proposed FTKD improves the model's ability to generalize to data diversity and local variability through two-stage knowledge distillation performs optimally.

In addition, from the accuracy trend graphs of the different methods in Fig. 8, it can be noticed that FedAvg and FedProx fluctuate more when dealing with non-IID datasets. This is because the core concept of these techniques is based on the joint efforts of all participants to maintain and update the same global model, but in practice, this approach reveals obvious limitations. When facing highly heterogeneous and decentralized data sources, this one-size-fits-all strategy is difficult to satisfy the unique needs and local characteristics of the respective subjects, leading to a significant reduction in model generalization capability and adaptability. Compared to FedAvg, FedYogi achieves faster convergence by adaptively adjusting the learning step size for each parameter. In comparison with other centralized federated learning methods, SCAFFOLD exhibits smaller oscillations in its test accuracy curve and converges more rapidly.

However, compared with all the above methods, the FTKD framework we propose demonstrates obvious advantages. It is not limited to patching a single problem, but precisely strengthens the ability of each client model to capture local unique data characteristics through the collaborative design of two-stage knowledge distillation, and effectively promotes the transfer and generalization of global knowledge. As a result, as shown in Fig. 8, FTKD outperforms methods such as FedAvg, FedProx, FedBN, FedYogi, and SCAFFOLD in terms of accuracy, conver-

Table 4
Performance of different methods on two datasets.

Methods	ToN-IoT				Edge-IIoTset			
	Accuracy (%)	Recall (%)	F1-Score (%)	Times (s)	Accuracy (%)	Recall (%)	F1-Score (%)	Times (s)
FedAvg	95.03	95.02	94.89	229.45	96.78	96.73	96.72	434.03
FedProx	96.46	96.38	96.32	272.48	97.19	97.18	97.12	576.90
FedBN	96.78	96.76	96.71	258.97	97.28	97.25	97.20	520.15
FedYogi	97.22	97.21	97.20	247.62	97.86	97.86	97.83	503.68
SCAFFOLD	97.24	97.24	97.22	256.35	98.14	98.14	98.11	529.58
FTKD(Ours)	97.72	97.72	97.71	263.70	98.69	98.69	98.64	529.58

Table 5
Performance of existing IDS methods in multiclass classification.

Datasets	Methods	FL framework	Accuracy (%)	Recall (%)	F1 (%)	Parameters	FLOPs
ToN-IoT	LKD-STNN	×	98.20	94.1	95.1	14,650	41,344
	CIDIoT	✓	86.36	77.75	79.08	43,929	-
	FEDSA	✓	95.77	95.77	95.77	-	-
	FTKD(Ours)	✓	97.72	97.72	97.71	2,860	62,241
Edge-IIoTset	DeepAK-IoT	×	94.96	-	94.72	-	-
	NASP-MTC	×	99.94	99.16	98.84	69,428	19×10^6
	SDN-Based	✓	95.41	-	-	7410	69,632
	FTKD(Ours)	✓	98.69	98.69	98.64	3,025	62,401

gence speed, and overall training performance. Particularly in terms of stability and robustness, FTKD demonstrates superior capabilities.

2) Comparison with state-of-the-art IDS methods

In addition to the comparison experiments with existing FL frameworks, we further compare with existing IDS methods. Table 5 shows the performance of our method relative to existing IDS methods when performing multi-classification tasks on the ToN-IoT and Edge-IIoTset datasets, where “-” denotes missing evaluation metrics in the original text.

From Table 5, we can find that our proposed FTKD demonstrates superiority over other federated learning IDS methods in ToN-IoT and Edge-IIoTset datasets. Although the centralized LKD-STNN approach slightly outperforms FTKD by 0.48 % in terms of accuracy, it falls short of FTKD in terms of Recall and F1-Score. This discrepancy stems from the fact that LKD-STNN fails to effectively deal with the problem of category imbalance. In addition, while the centralized NASP-MTC approach is more than 0.2 % higher in terms of accuracy, Recall, and F1-Score, its model parametric counts and FLOPs are significantly higher than our FTKD approach. For resource-constrained IIoT devices, this high demand of computational resources makes NASP-MTC difficult to deploy and apply. In contrast, our proposed FTKD approach pays special attention to model efficiency and lightweight design, ensuring its applicability in resource-limited environments. Notably, even when facing the challenges of non-IID data, our approach still achieves performance close to that of centralized methods, which fully demonstrates the effectiveness and robustness of FTKD. Thus, FTKD provides a more feasible option for deploying an efficient intrusion detection system in real IIoT environments, which guarantees good detection performance while considering the effective utilization of hardware resources.

6.4. Privacy analysis

Based on the definition of the DP method, we know that the level of data privacy protection is closely related to the degree of perturbation applied. To assess the data privacy after DP treatment, we can do so by analyzing the similarity between the perturbed weights and the original weights. For this purpose, we will use as a measurement tool the Pearson correlation coefficient (PCC), a statistical measure of the strength and direction of the linear relationship between two variables with a correlation coefficient $\rho \in [-1, 1]$. Essentially, the PCC is the ratio between the covariance and standard deviation of two variables. Its calculation

Table 6
Pearson correlation coefficient and performance.

ϵ	Mechanism	PCC	Accuracy (ToN)	Accuracy (Edge)	MIA(Accuracy)
0.1	Laplace	0.3673	97.63 %	98.42 %	0.7949
0.5	Laplace	0.5297	97.69 %	98.48 %	0.8423
1	Laplace	0.7367	97.71 %	98.53 %	0.8967
0.1	Gaussian	0.2824	97.65 %	98.39 %	0.7798
0.5	Gaussian	0.5142	97.68 %	98.45 %	0.8440
1	Gaussian	0.6891	97.69 %	98.49 %	0.8813
0.1	Staircase	0.1646	97.43 %	98.26 %	0.7423
0.5	Staircase	0.3601	97.55 %	98.41 %	0.8033
1	Staircase	0.5219	97.70 %	98.47 %	0.8576
0	Uniform	0.1929	97.67 %	98.46 %	0.7417

formula can be expressed as:

$$\rho = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}} \quad (68)$$

where x_i is the *Logits* before DP noise addition and y_i is the *Logits* after DP noise addition.

However, PCC primarily reflects the linear correlation between variables. While it can indirectly indicate the strength of perturbations, it struggles to directly demonstrate the robustness of the model in real-world privacy attack scenarios or the reliability of the uniform perturbation mechanism. To address this, we further introduce Member Inference Attack (MIA) as an empirical privacy evaluation method to more directly assess the effectiveness of differential privacy protection based on the uniform perturbation mechanism. The experiments adopt the standard MIA framework proposed by Shokri et al. (2017b), which constructs multiple shadow models (i.e., client models simulating the training process) to approximate the behavior of the real model. Each shadow model (simple CNN network) is trained on a subset of data from a public dataset and generates predicted *Logits* as member outputs. Subsequently, the trained real client generates predicted *Logits* as non-member outputs on another subset of data from the public dataset. Finally, these *Logits* and their member labels are used to train a lightweight attack model (simple CNN network) to determine whether the input samples corresponding to the target model’s outputs belong to its training set.

To highlight the effectiveness of the uniform perturbation mechanism in achieving differential privacy, Table 6 lists the average PCC

values, model accuracy and MIA attack accuracy for different perturbation mechanisms and different values of ϵ . From Table 6, shows that the uniform mechanism provides a relatively fuzzified local *Logits*, thus achieving a lower PCC value. This means that the uniform mechanism can provide a higher level of privacy protection among all the differential privacy techniques analyzed in this study. Furthermore, despite the strong privacy protection, the uniform mechanism maintains a level of accuracy comparable to other DP techniques.

In this evaluation, we further focus on the attack model's ability to infer client model outputs under different privacy budgets. Experimental results show that, without DP, the attack model achieves a high inference accuracy of 90 %, indicating a clear risk of privacy leakage from the raw logits. However, the accuracy falling short of 100 % suggests that the FTKD-trained client models possess a certain level of generalization, mitigating perfect membership inference. After applying DP mechanisms with Gaussian and Laplace perturbations, the inference accuracy drops significantly. Notably, when employing the proposed uniform perturbation mechanism, the MIA attack accuracy decreases by an additional 16 % compared to other DP methods. This result strongly validates the effectiveness of our approach in defending against practical privacy attacks, demonstrating that the uniform perturbation not only reduces information leakage at the statistical level but also enhances the model's resilience against advanced inference attacks at the semantic level.

7. Conclusion

In this paper, we propose a federated learning framework based on two-stage knowledge distillation, which aims to address the multiple challenges faced by IDS for intrusion detection systems in IIoT environments. By combining lightweight model design with advanced knowledge distillation techniques, our proposed FTKD effectively improves the performance of intrusion detection. First, the FTKD approach ensures efficient intrusion detection on resource-constrained IIoT devices by designing a lightweight MA-TBCLNN model, and enhances the personalized learning capability of the local model through the APSKD and PKD policies, ensuring that different clients can optimize the model performance based on their own data characteristics. To address the problem of non-IID data, FTKD introduces an aggregation strategy based on Shannon entropy and maximum mean difference (EMMD) to dynamically adjust the model weights, thus effectively eliminating the training bias due to uneven data distribution. Finally, to protect user privacy, FTKD adopts a differential privacy protection mechanism, utilizes the model-predicted probability distribution *Logits* as the aggregation object, and introduces a uniform perturbation mechanism to reduce the risk of sensitive information leakage. Experimental results show that the FTKD method achieves greater than 97 % accuracy on both ToN-IoT and Edge-IIoTset datasets, which significantly outperforms existing federated learning methods and most of the centralized intrusion detection methods, and verifies the effectiveness and robustness of FTKD in dealing with complex network security threats.

CRedit authorship contribution statement

Renqiang Zhou: Writing – original draft, Methodology, Visualization, Software, Conceptualization ; **Zhendong Wang:** Writing – review & editing, Funding acquisition, Formal analysis, Validation; **Shuxin Yang:** Supervision, Visualization; **Daojing He:** Visualization, Investigation; **Sammy Chan:** Resources, Supervision.

Data availability

Data will be made available on request.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the Natural Science Foundation of China (62062037, 61562037, 72261018), the Natural Science Foundation of Jiangxi Province (20212BAB202014, 20171BAB202026).

References

- Ahmad, B., Wu, Z., Huang, Y., & Rehman, S. U. (2024). Enhancing the security in iot and IIoT networks: An intrusion detection scheme leveraging deep transfer learning. *Knowledge-Based System*, 305, 112614.
- Borgwardt, K. M., Gretton, A., Rasch, M. J., Kriegel, H.-P., Schölkopf, B., & Smola, A. J. (2006). Integrating structured biological data by kernel maximum mean discrepancy. *Bioinformatics*, 22(14), e49–e57.
- Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010: 19th international conference on computational statistics, paris, france, august 22–27, 2010 keynote, invited and contributed papers* (pp. 177–186). Springer.
- Ding, W., Abdel-Basset, M., & Mohamed, R. (2023). DeepAK-iot: An effective deep learning model for cyberattack detection in iot networks. *Information Sciences*, 634, 157–171.
- Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(7).
- Ferrag, M. A., Friha, O., Hamouda, D., Maglaras, L., & Janicke, H. (2022). Edge-IIoTset: A new comprehensive realistic cyber security dataset of iot and IIoT applications for centralized and federated learning. *IEEE Access*, 10, 40281–40306.
- Fredrikson, M., Jha, S., & Ristenpart, T. (2015). Model inversion attacks that exploit confidence information and basic countermeasures. In *Proc. ACM conf. computer commun. secur.* (pp. 1322–1333).
- Geng, Q., Ding, W., Guo, R., & Kumar, S. (2018). Privacy and utility tradeoff in approximate differential privacy. *arXiv preprint arXiv:1810.00877*.
- Han, W., Peng, J., Yu, J., Kang, J., Lu, J., & Niyato, D. (2023). Heterogeneous data-aware federated learning for intrusion detection systems via meta-sampling in artificial intelligence of things. *IEEE Internet of Things Journal*.
- He, K., Kim, D. D., & Asghar, M. R. (2023). Adversarial machine learning for network intrusion detection systems: A comprehensive survey. *IEEE Communications Surveys and Tutorials*, 25(1), 538–566.
- Hinton, G. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Kairouz, P., McMahan, H. B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A. N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R. et al. (2021). Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2), 1–210.
- Karimireddy, S. P., Kale, S., Mohri, M., Reddi, S., Stich, S., & Suresh, A. T. (2020). Scaffold: Stochastic controlled averaging for federated learning. In *International conference on machine learning* (pp. 5132–5143). PMLR.
- Kim, K., Ji, B., Yoon, D., & Hwang, S. (2021). Self-knowledge distillation with progressive refinement of targets. In *Proc. IEEE int. conf. comput. vision* (pp. 6567–6576).
- Li, S., Cao, Y., Liu, S., Lai, Y., Zhu, Y., & Ahmad, N. (2024). Hda-ids: A hybrid dos attacks intrusion detection system for iot by using semi-supervised cl-gan. *Expert Systems with Applications*, 238, 122198.
- Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks. *Proceedings of Machine Learning and Systems*, 2, 429–450.
- Li, X., Jiang, M., Zhang, X., Kamp, M., & Dou, Q. (2021). Fedbn: Federated learning on non-iid features via local batch normalization. *arXiv preprint arXiv:2102.07623*.
- Liu, M., Xue, Z., Xu, X., Zhong, C., & Chen, J. (2018). Host-based intrusion detection system with system calls: Review and future trends. *ACM Computing Surveys*, 51(5), 1–36.
- McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. In *Proc. int. conf. artif. intell. stat.* (pp. 1273–1282). PMLR.
- Mothukuri, V., Khare, P., Parizi, R. M., Pouriyeh, S., Dehghantanha, A., & Srivastava, G. (2021). Federated-learning-based anomaly detection for iot security attacks. *IEEE Internet of Things Journal*, 9(4), 2545–2554.
- Moustafa, N. (2021). A new distributed architecture for evaluating AI-based security systems at the edge: Network TON iot datasets. *Sustainable Cities and Society*, 72, 102994.
- Moustafa, N., Koroniotis, N., Keshk, M., Zomaya, A. Y., & Tari, Z. (2023). Explainable intrusion detection for cyber defences in the internet of things: Opportunities and solutions. *IEEE Communications Surveys and Tutorials*, 25(3), 1775–1807.
- Nandanwar, H., & Katarya, R. (2024). Deep learning enabled intrusion detection system for industrial IOT environment. *Expert Systems with Applications*, 249, 123808.
- Qu, Z., & Cai, Z. (2024). Fedsa-resnetv2: An efficient intrusion detection system for vehicle road cooperation based on federated learning. *IEEE Internet of Things Journal*.
- Rahman, S. A., Tout, H., Talhi, C., & Mourad, A. (2020). Internet of things intrusion detection: Centralized, on-device, or federated learning? *IEEE Network*, 34(6), 310–317.
- Reddi, S., Charles, Z., Zaheer, M., Garrett, Z., Rush, K., Konečný, J., Kumar, S., & McMahan, H. B. (2020). Adaptive federated optimization. *arXiv preprint arXiv:2003.00295*.

- Ross, T.-Y., & Dollár, G. (2017). Focal loss for dense object detection. In *Proc. IEEE int. conf. comput. vision* (pp. 2980–2988).
- Shao, H., Xiao, Z., Yao, S., Sun, D., Zhang, A., Liu, S., Wang, T., Li, J., & Abdelzaher, T. (2021). ControlVAE: Tuning, analytical properties, and performance analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(12), 9285–9297.
- Shokri, R., Stronati, M., Song, C., & Shmatikov, V. (2017a). Membership inference attacks against machine learning models. In *Proc. IEEE symp. secur. privacy* (pp. 3–18). IEEE.
- Shokri, R., Stronati, M., Song, C., & Shmatikov, V. (2017b). Membership inference attacks against machine learning models. In *2017 IEEE Symposium on security and privacy (SP)* (pp. 3–18). IEEE.
- Wang, L.-H., Dai, Q., Du, T., & Chen, L.-f. (2024a). Lightweight intrusion detection model based on CNN and knowledge distillation. *Applied Soft Computing*, 165, 112118.
- Wang, Z., Li, J., Yang, S., Luo, X., Li, D., & Mahmoodi, S. (2024b). A lightweight iot intrusion detection model based on improved BERT-of-theseus. *Expert Systems with Applications*, 238, 122045.
- Wang, Z., Zhou, R., Yang, S., He, D., & Chan, S. (2025). A novel lightweight iot intrusion detection model based on self-knowledge distillation. *IEEE Internet of Things Journal*, 12(11), 16912–16930.
- Xenofontos, C., Zografopoulos, I., Konstantinou, C., Jolfaei, A., Khan, M. K., & Choo, K.-K. R. (2022). Consumer, commercial, and industrial iot (in)security: Attack taxonomy and case studies. *IEEE Internet of Things Journal*, 9(1), 199–221.
- Yang, S., Yang, J., Zhou, M., Huang, Z., Zheng, W.-S., Yang, X., & Ren, J. (2024). Learning from human educational wisdom: A student-centered knowledge distillation method. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Yeom, S., Giacomelli, I., Fredrikson, M., & Jha, S. (2018). Privacy risk in machine learning: Analyzing the connection to overfitting. In *Proc. IEEE comput. secur. found. symp.* (pp. 268–282). IEEE.
- Zainudin, A., Akter, R., Kim, D.-S., & Lee, J.-M. (2023). Federated learning inspired low-complexity intrusion detection and classification technique for sdn-based industrial cps. *IEEE Transactions on Network and Service Management*.
- Zhang, J., Luo, C., Carpenter, M., & Min, G. (2022). Federated learning for distributed IIoT intrusion detection using transfer approaches. *IEEE Transactions on Industrial Informatics*, 19(7), 8159–8169.
- Zhang, X., Hao, L., Gui, G., Wang, Y., Adebisi, B., & Sari, H. (2023). An automatic and efficient malware traffic classification method for secure internet of things. *IEEE Internet of Things Journal*.
- Zhao, R., Wang, Y., Xue, Z., Ohtsuki, T., Adebisi, B., & Gui, G. (2022). Semisupervised federated-learning-based intrusion detection method for internet of things. *IEEE Internet of Things Journal*, 10(10), 8645–8657.
- Zhu, S., Xu, X., Zhao, J., & Xiao, F. (2023). Lkd-stnn: A lightweight malicious traffic detection method for internet of things based on knowledge distillation. *IEEE Internet of Things Journal*.